

MUNARIUM LAB · APPLIED AI GOVERNANCE SERIES

Munarium Governance Platform

An Open Source Applied AI Journey

Why a governed applied AI platform is being built in the open, how it is designed to work, and how its founder is building it with bounded coding agents in VCP

SERIES	Munarium Lab · Applied AI Governance Series
DOCUMENT	Business whitepaper
STATUS	• Forward-looking • Describes planned open-source work • Nine new components are at “repository created”
AS OF	29 September 2026
PUBLISHER	Ioka LLC · ioka.io · Mapleton, Utah
PLANNING HUB	github.com/iokaio/munarium-platform (Apache-2.0)

Forward-looking statement: This paper describes intended architecture and planned work. Where it differs from a repository, the repository's README and evidence are authoritative. Case studies are synthetic and illustrative. Nothing here is a certification, a compliance determination or a support commitment.

IN BRIEF**Let agents propose.****Keep authority, evidence and rules somewhere else.**

AI agents are moving from answering to acting, and most early deployments give them the same credentials, approval paths and rule-changing rights as the systems they call. The **Munarium Governance Platform** separates four powers, **read, governed write, act and govern**, across four planes and eleven open-source services, so an agent can do real work without holding target credentials, approving its own effects or activating the policy that governs it.

The foundation, Munarium Server and Matrix, is released today. Nine new components are public under Apache-2.0 and planned in dependency order toward a twelve-month reference platform. The founder is building them with **VCP** and other coding agents under the same discipline the product enforces: agents propose and implement; a protected, human-held path accepts and releases.

- Why: five governance gaps that agency opens, and why memory governance alone cannot close them (Section 2).
- How it works: the architecture, the nine-step governed action and consequence classes (Sections 4 to 8).
- How it is built: the hub, contracts, invariants, roadmap and the VCP development method (Sections 9 and 10).
- How to start: three synthetic case studies and a twelve-action playbook you can use now (Sections 12 and 13).

Contents

1	Executive summary	5
2	Why this effort: the applied AI governance gap	6
2.1	Agency changes the risk.....	6
2.2	Why governed memory was necessary but not sufficient.....	7
2.3	The governed must not govern the governor	7
2.4	Why build it in the open, and why now	7
3	The journey: from governed memory to governed action.....	8
3.1	A platform that grew from a practical problem.....	8
3.2	An AI-assisted organization of one	9
3.3	Honesty as a product feature	9
4	Design principles in business terms	10
4.1	Four powers, deliberately separated	10
4.2	Principles and what they mean for the business	10

4.3 A bounded guarantee	11
5 How the platform works	11
5.1 Four planes and eleven services	11
5.2 The components at a glance	13
5.3 The foundation you can use today	13
6 A governed action, end to end	14
6.1 One request, several distinct decisions	14
6.2 Why unresolved is an honest outcome	15
6.3 Consequence classes set the level of control	15
6.4 Provenance is evidence, not a magic taint tracker	15
6.5 Behavior when dependencies fail	16
7 Governing change itself	16
7.1 Propose, ratify, activate	16
7.2 Weakening a control is itself a governance change	17
7.3 Solving the bootstrap problem honestly	17
8 Evidence and assurance	17
8.1 Sentinel: useful before sophisticated	18
8.2 Assure: evidence that can be checked elsewhere	18
8.3 Console: one interface, no hidden privilege	18
8.4 Alignment with the frameworks your auditors use	18
9 The development perspective	19
9.1 One hub, nine repositories	19
9.2 Contracts before code	20
9.3 Invariants as the acceptance language	20
9.4 The founder-led roadmap	20
9.5 What it costs to build in the open	21
10 Building it with VCP	22
10.1 What VCP is	22
10.2 How VCP will be used	22
10.3 Why this method: the product and the process are the same idea	23
10.4 Guardrails on the method itself	24
11 Adoption: meeting enterprises where they are	24
11.1 Maturity is not a prerequisite	24
11.2 Brownfield patterns	25
11.3 Many stacks, selective qualification	26
12 Three forward-looking case studies	26
Case A. Larkspur Software: coding agents and a controlled release path	28

Case B. Kestrel Valley Credit Union: vendor maintenance before payment	29
Case C. Aspen Crest Health Partners: scheduling and patient messaging	31
What the three cases have in common.....	32
13 Getting a jump on applied AI governance	32
13.1 A ninety-day starter plan	33
14 How to engage with the open-source program.....	34
15 Conclusion.....	34
Appendix A. The invariant catalog as a checklist.....	36
Appendix B. Working glossary	37
Appendix C. Sources.....	37

1 Executive summary

Enterprises are moving from AI that answers questions to AI that takes actions. A research assistant becomes an agent that updates a vendor record, opens a pull request, schedules a patient, or releases a build. That shift changes the governance question. It is no longer enough to ask whether a model is accurate or whether a prompt is appropriate. Leaders now need to know who allowed an effect, on what evidence, under which rule, and whether the agent could have granted that permission to itself.

The **Munarium Governance Platform** is Ioka LLC's answer to that question, built entirely in the open. It extends Munarium's released governed-memory foundation, Munarium Server and Munarium Matrix, with nine new open-source components that separate what an agent may **read**, what it may **write** through governed checks, what it may **do** in the world, and who may **govern** the rules themselves. Agents may reason, gather evidence and propose. Their ordinary operating identity may not hold the credential that acts on a production system, approve its own consequential effects, or activate the policy that constrains it.

This paper explains the reasons behind the effort, how the platform is designed to work from a business and high-level development perspective, and how the founder is building it with **Vibe Code Pro (VCP)**, an open-source coding environment, together with other coding agents under the same bounded discipline the platform enforces. It closes with three synthetic, forward-looking case studies and a practical playbook that any organization can start on now, whether or not it ever deploys Munarium.

Five things to take away

- **Governance has to sit on the path, not beside it.** Policies that live in documents or prompts do not stop an agent that holds a live credential. Controls belong where the consequential call is actually made.
- **Separate four powers.** Read, governed write, act and govern are different permissions. Most early agent deployments collapse them into one service account.
- **Evidence is a product feature.** Every allowed, denied, approved or unresolved action should leave a record that someone outside the system can verify.
- **Start narrow and prove the boundary.** One high-value workflow, observed first and then enforced, teaches more than an enterprise-wide policy rollout.
- **Build the way you govern.** The platform is being built by one accountable founder with coding agents that propose and implement, but never release, the same separation the product sells.

The platform is early. As of 29 September 2026 the architecture hub and nine component repositories are public under Apache-2.0 with design guides and interface scaffolds; every component's functional capability is **planned**, and no platform release exists yet. The

twelve-month target is a coherent, single-cell reference platform with a documented path from an agent's proposal to an independently authorized effect. This paper is candid about that status because candor about boundaries is the platform's central design commitment.

2 Why this effort: the applied AI governance gap

2.1 Agency changes the risk

The first wave of enterprise generative AI was largely read-only. Chat assistants summarized documents, drafted text and answered questions, and the principal risks were accuracy, disclosure and cost. Governance programs responded with acceptable-use policies, model approval lists, prompt reviews and output filters. Those controls still matter, but they were designed for a system that talks.

Agentic systems act. They call tools, invoke APIs, open tickets, change records and delegate work to other agents. In many early deployments they do so through a service account or a developer token that already has broad authority over the target system. When that happens, every safeguard that lives inside the agent, whether a system prompt, a framework callback or a model's own judgment, becomes the last line of defense. A single prompt injection hidden in an invoice, a web page or a support ticket can turn a helpful agent into a confident operator of the wrong instruction.

THE GOVERNANCE GAP

Five gaps that agency opens, and the control that closes each

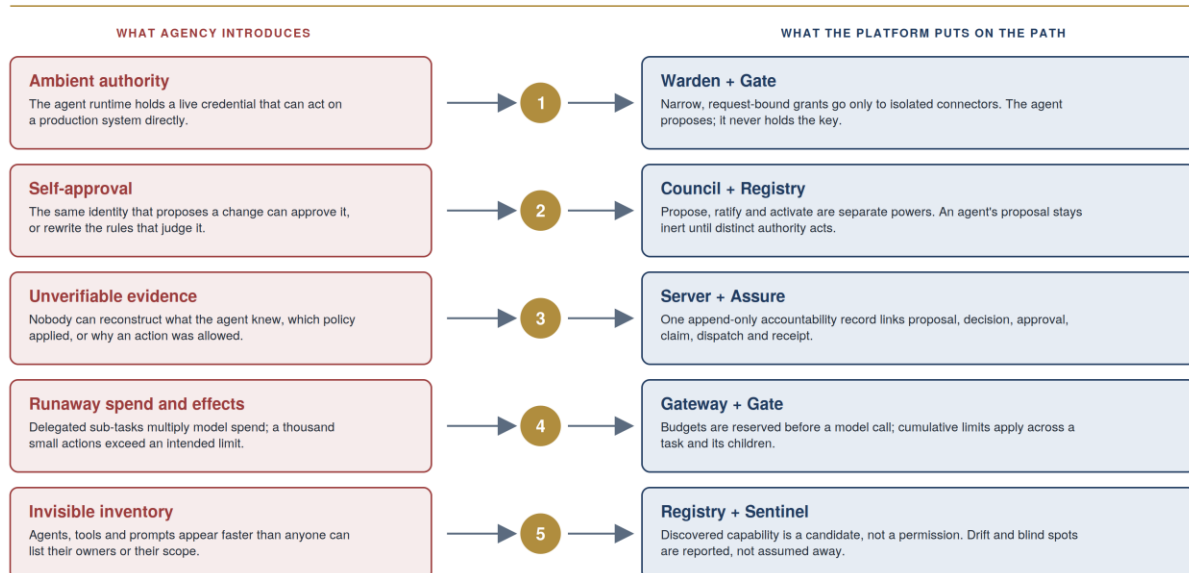


Figure 1. Five governance gaps that agency introduces, each mapped to the platform control designed to close it.

Figure 1 summarizes the five gaps the platform is designed to close. They share a pattern: each is a question of **authority** rather than intelligence. A smarter model does not remove

the need for a separate approval, a narrower credential or a verifiable record. It usually increases the volume of work that needs them.

2.2 Why governed memory was necessary but not sufficient

Munarium began as governed memory. Munarium Server keeps an append-only ledger of claims with their disputes, corrections, supersession and provenance, so an AI application can show which evidence supported an answer and how that evidence changed over time. Munarium Matrix extends the same discipline to structured enterprise data through reviewed, read-only query contracts that return sealed evidence. Together they address a real problem: plausible language is not the same as a supported claim.

Working on that foundation made a second boundary obvious. A reliable memory layer cannot neutralize an unrestricted production credential sitting in the application around it. The evidence plane and the authority to create effects are different responsibilities. An agent can be perfectly well informed and still be the wrong party to approve a payment, deploy a build or change the rules that judge its own behavior.

2.3 The governed must not govern the governor

The founder's companion position paper states the principle that organizes the whole platform: **the governed must not govern the governor**. An agent may propose a change to the rules that constrain it. It must not be able to ratify or activate that change through its ordinary identity, directly or through a delegated task, a modified configuration or a substituted artifact. The same principle applies to effects: an agent may draft a payment, but the payment credential lives elsewhere, and a distinct authority approves the exact request.

THE GOVERNING DELIVERY RULE

Narrow the release; never waive the control

Every stage of the platform must be useful and honest within its stated boundary. Missing authority, unavailable evidence, incomplete credential isolation or an unqualified connector narrows what a release claims. None of them is ever a reason to waive a control to hit a date.

2.4 Why build it in the open, and why now

Governance infrastructure asks for trust, and trust is cheaper to earn when the controls can be inspected. An architect evaluating the platform should be able to read an action contract, run a denial test, see the code that checks an approval and reproduce the decision trace on a laptop. The strongest sales asset for a control plane is a small working example whose boundary can be tested, not a large diagram whose boxes remain hypothetical.

Openness also makes limitations visible. A connector can publish its declared scopes and failure modes. A release can show which authority checks are implemented and which are not. Independent teams can evaluate those claims without relying on the vendor's explanation. For that reason all nine new components are Apache-2.0 from their first public commit, and there is no code-based community-versus-enterprise split among them.

Deferred is a roadmap state, not a commercial restriction.

The timing matters because organizations are making architecture decisions about agents now, often by default. Credentials granted to a pilot tend to survive into production. Approval habits formed around chat assistants get stretched over agents that act. Regulatory expectations, including the EU AI Act's record-keeping and human-oversight provisions and management-system standards such as ISO/IEC 42001, reward organizations that can produce evidence rather than assurances. Getting the separation of powers right early is far cheaper than retrofitting it after an incident.

3 The journey: from governed memory to governed action

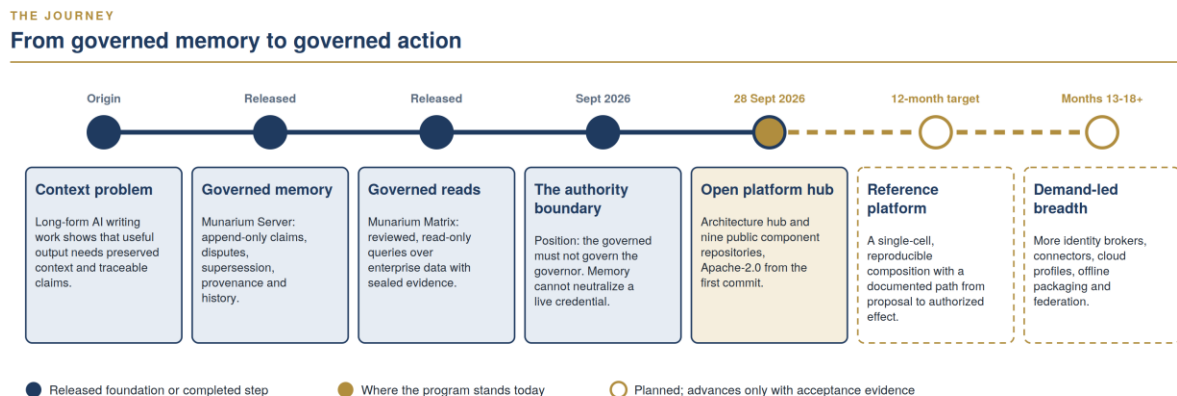


Figure 2. The path from a practical context problem to an open governance platform. Released steps are shown in navy; planned steps advance only with evidence.

3.1 A platform that grew from a practical problem

The platform did not start as an enterprise-governance strategy. It started with long-form AI-assisted writing, where useful output depends on preserving context, distinguishing supported claims from plausible ones and keeping the evidence behind each change. Those same needs reappeared, at higher stakes, in enterprise memory and retrieval work, and they produced Munarium Server and Matrix. The Governance Platform is the next architectural response to the same problem as AI systems move from recalling to acting.

3.2 An AI-assisted organization of one

The founder built the existing foundation himself, with the help of coding agents and multiple models, while retaining responsibility for architecture, verification and release. The platform plan continues that practice deliberately. There is no recruiting period or financing event on the critical path. Optional reviewers, sponsors, contributors and design partners can strengthen the project, but they are not counted as delivery capacity.

That choice makes the scarce resource explicit. It is not code generation; coding agents can produce more code than one person can responsibly review. It is **founder attention**: the time to choose a sound design, inspect a difficult change, investigate an ambiguous failure, maintain released software and respond to adopters. The delivery model, described in Section 10, is organized around protecting that attention.

3.3 Honesty as a product feature

A governance platform that overstates its own guarantees is worse than none, because it invites reliance that has not been earned. The project therefore uses evidence labels rather than marketing tiers. Each component, and the platform composition as a whole, advertises only what its evidence supports, and the composition can never claim a stronger boundary than its weakest required dependency.

Table 1. Release labels describe evidence, not editions

Label	Minimum meaning
Planned	Architecture or work items exist; no functioning capability is claimed.
Experimental	A runnable prototype exists, with explicit limitations and no broad production claim.
Conformance-tested	A named version passes the published suite for a stated environment and contract.
Reference-qualified	The integrated composition passes operational, recovery, authority and deployment tests for a specific profile.
Independently reviewed	A named independent review covers a stated revision and scope, with findings and residual limits recorded.

Today every new component sits at the catalog state **repository created**, and every field in the platform's twenty-two-item invariant catalog is blank, which the project defines as unverified rather than passed. That is an unusual thing for a vendor to publish. It is also the clearest possible signal of how the platform intends to behave when it matters.

4 Design principles in business terms

4.1 Four powers, deliberately separated

The conceptual center of the platform is a separation of four powers. A principal, whether a person, a service or an agent, may hold one without holding the others. The separation is what allows an organization to give agents meaningful work without giving them the keys.

Table 2. The four powers and where each is mediated

Power	What it means	Where it is mediated	The agent's part
Read	Retrieve permitted evidence from memory or enterprise data	Server, Matrix	Scoped access
Governed write	Submit claims or corrections that pass governance checks	Server	Scoped proposal
Act	Create a bounded effect in an external system	Gate, Warden	Proposal only
Govern	Activate the rules, manifests and profiles that govern everything else	Council, Registry	No activation

The revised plan adds an important refinement: **proposing a governance change is different from activating governance**. An agent can submit an inert candidate through a narrow intake surface; that does not make it a writer to the active authority state. It also names a practical trap. Two credentials controlled by the same person do not create two independent reviewers, and two services sharing one administrator account are not independent because their process names differ. A deployment has to describe who can actually alter each boundary.

4.2 Principles and what they mean for the business

Table 3. Platform principles translated into business consequences

Principle	What it means for an adopting organization
Agent code is untrusted	SDKs and prompts are conveniences. Enforcement lives outside the agent process, so a compromised or manipulated agent still cannot exceed its grant.
Controls sit on consequential paths	A direct credential, network route or vendor connector outside mediation is a documented bypass, not a hidden one. You know what is covered.
Default deny, fail closed	Unknown tools, unverifiable approvals, missing mandatory evidence and policy errors produce refusals, not permissions.
Decisions are reproducible	Every decision is made over pinned inputs and a recorded policy version, so an auditor can replay why something was allowed.

Principle	What it means for an adopting organization
Target secrets stay out of view	The identity that authenticates an agent is never the credential that can act directly on a target system.
Provenance influences policy	Cited evidence is verified. Missing lineage is treated as unknown, never as implicitly authoritative.
One accountability record	Dashboards and caches are rebuildable views of a single record, not competing histories that disagree during an incident.
Integrate mature infrastructure	Munarium does not become another identity provider, secrets vault, ticketing suite or SIEM. It uses the ones you have.

4.3 A bounded guarantee

The platform's guarantee is deliberately conditional. Within a qualified deployment, the specified consequential path is mediated, and the agent cannot acquire through that path any authority the deployed controls deny. A separate credential, an unmanaged host, an unprotected approval pipeline or a target-side rule outside the tested boundary can invalidate that claim. The platform will say so in its evidence exports rather than implying uniform coverage. For business owners, that is the difference between a control you can rely on and a control you merely hope applies.

5 How the platform works

5.1 Four planes and eleven services

The platform composes into four trust planes. They are responsibility boundaries, not a requirement for eleven separate servers; a small reference deployment may run several services on one machine as long as processes, credentials, network permissions and storage roles stay separate. Figure 3 shows the planes, the components in each and the interactions that produce a governed platform.

PLATFORM ARCHITECTURE

Four planes, eleven services, one accountability record

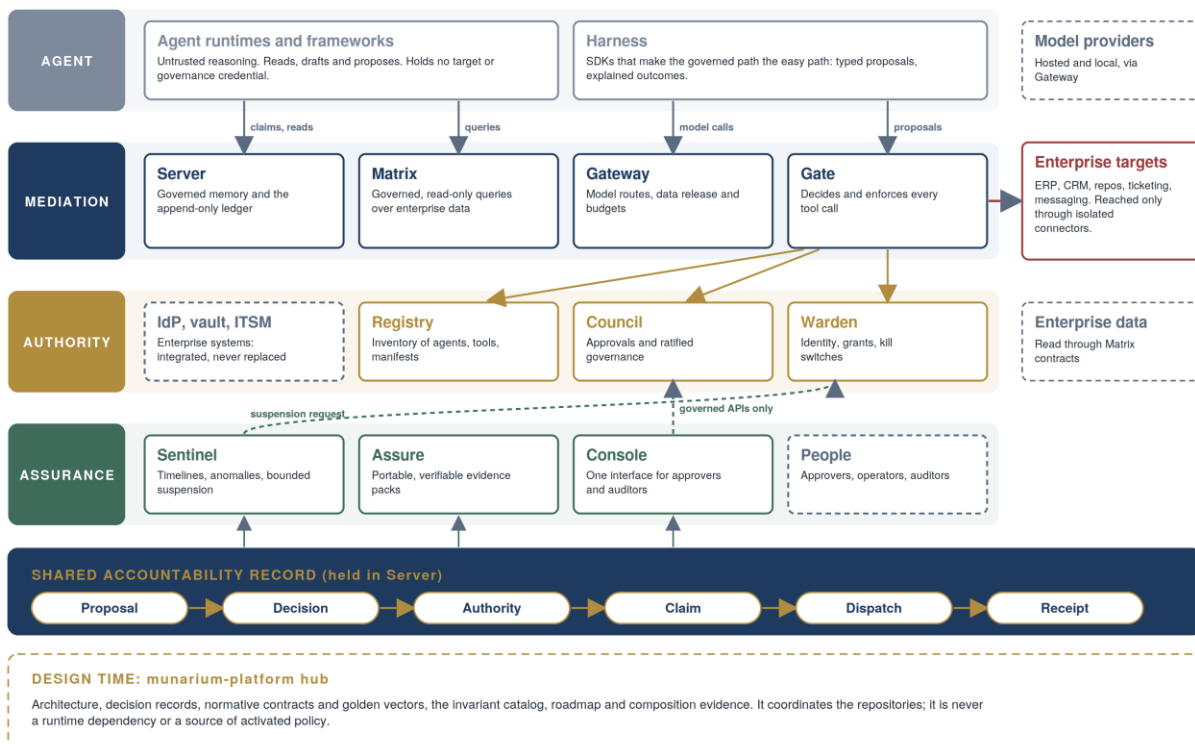


Figure 3. The platform architecture. Agents reach models, memory, data and actions only through mediation services; authority and assurance components never execute target operations themselves.

The **agent plane** contains application runtimes, agent frameworks and the Harness SDKs. Its code can reason, read within scope and propose. It cannot inherit cloud administrator credentials, read broker secrets, edit active policy or use an unrestricted network route to reach a consequential target.

The **mediation plane** is where requests become governed. Gate evaluates and enforces tool calls; Gateway mediates model invocations, including the data they disclose and the budget they consume; Server records governed memory and the accountability ledger; Matrix answers reviewed, read-only queries over enterprise data. A read can disclose restricted records and a model call can move data, so each path is authorized according to its actual consequence, not its HTTP method.

The **authority plane** decides who may do what. Registry holds the approved inventory of agents, tools, manifests and policy bundles. Warden verifies identity and delegation and issues narrow execution grants. Council records approvals bound to exact requests and runs the lifecycle of governance changes. No ordinary agent credential can modify their effective state.

The **assurance plane** makes the system inspectable. Sentinel builds timelines and telemetry and can request pre-authorized suspension. Assure packages portable, verifiable

evidence. Console gives approvers, operators and auditors one interface, and it calls the same governed APIs as any other client, with no hidden administrative path.

5.2 The components at a glance

Table 4. The nine new components and their first useful increments

Component	Plane	What it does for the business	First useful public increment
Registry	Authority	Turns an agent and tool inventory into something enforceable: discovered capability stays inert until activated.	Signed manifest catalog and schema validation
Harness	Agent	Makes the governed path the easiest path for developers, with typed proposals and explained outcomes.	Typed proposal and outcome client with examples
Warden	Authority	Connects a verified actor to a narrowly bounded capability; delivers kill switches that actually stop new work.	One verified identity path and one isolated broker
Gate	Mediation	Decides and enforces every tool call, records durable claims, dispatches through isolated connectors.	Replayable decision engine and fake-target lifecycle
Gateway	Mediation	Controls model routes, data release and spend, with budgets reserved before a call is made.	One hosted and one local provider path with receipts
Council	Authority	Binds approvals to exact requests and separates proposal, ratification and activation of rules.	Request-bound approval and immutable activation
Sentinel	Assurance	Reconstructs what happened, surfaces blind spots and requests bounded suspension.	Ledger-derived timeline and authenticated suspension
Assure	Assurance	Produces evidence packs an auditor can verify offline, with declared omissions.	Portable evidence pack and offline verifier
Console	Assurance	One interface for approvers, operators and auditors, with no privilege the APIs lack.	Read-only inventory and decision views

5.3 The foundation you can use today

Two parts of the platform are already released software with their own evidence and support boundaries. **Munarium Server** (1.3.0) provides governed memory: append-only claims, disputes, historical reads, capability tokens, governance profiles and provider-call accounting. **Munarium Matrix** (1.2.0) provides a governed read path to enterprise databases, warehouses and semantic layers through verified query contracts, sealed evidence and authorization classes. Organizations that want a head start on applied AI governance can adopt both now for assistant and research workloads, where the most important control is knowing which evidence supported which answer.

The platform plan identifies nine upstream Server changes that the action path needs, from separating governance authority from ordinary write roles to factoring a shared guarded-execution library. The first, separating governance activation from governed writes with an explicit, non-agent bootstrap path, is scheduled before anything else, because every later guarantee depends on it.

6 A governed action, end to end

6.1 One request, several distinct decisions

A governed action begins with a **proposal, not a credential**. Figure 4 traces the nine steps between an agent's request and a checkable outcome. Each step records its authoritative transition when it happens, not only after the workflow succeeds, so a failure halfway through leaves a truthful partial record and a recoverable state.

THE GOVERNED ACTION

Nine steps from an agent's proposal to checkable evidence



Figure 4. The governed action lifecycle. Required records and a durable claim precede any grant or dispatch; uncertain outcomes enter investigation rather than retry.

In business terms, the sequence answers the questions an auditor, an incident responder or a board member will eventually ask. **Who asked?** Warden's verified principal chain, including any delegation, rather than an identity the agent reported about itself. **Was it allowed?** Gate's deterministic decision over a signed manifest and pinned inputs, replayable later. **Who approved it?** Council's approval, bound to the exact request so that a changed recipient, amount, artifact or target invalidates it. **What actually happened?** The receipt, or an explicit unresolved state when the remote system's outcome is unknown.

6.2 Why unresolved is an honest outcome

Distributed systems lose responses. A payment API can apply a request and time out before confirming it. Many automation stacks treat that as a failure and retry, which is how duplicate payments, duplicate tickets and duplicate deployments happen. The platform does not promise exactly-once external effects, because no platform can. Instead it preserves the ambiguity, stops blind repetition and routes the item to investigation. A compensation, such as reversing a transaction, is a new governed action with its own authorization and record.

6.3 Consequence classes set the level of control

CONSEQUENCE CLASSES

Controls scale with what an action can do, not with how it is described

Everyday examples:

C0 summarize approved policy documents for an employee
C1 create an internal draft ticket from a support thread

Modifiers only raise the class

Amount, recipient, environment, data sensitivity and weak evidence push a request upward. An agent calling an action harmless never lowers it.



Figure 5. Consequence classes C0 to C4 and their default control postures. The class is computed from the approved manifest and verified context, never from the agent's description.

Not every action deserves the same friction. The platform classifies each capability from C0 (bounded observation) to C4 (prohibited for the requesting principal). Modifiers such as amount, recipient, environment, data sensitivity and evidence quality can raise the class but never lower it, and cumulative limits catch the thousand small actions that together exceed an intended boundary. Classification is contextual: a draft can reveal confidential material, and a read that sends protected records to an external model is not low-consequence simply because it is a read.

6.4 Provenance is evidence, not a magic taint tracker

Untrusted content must not become authority merely by appearing in a model's context. Consider an accounts-payable agent that receives an invoice announcing new bank details. The email is retained as external evidence, but its content supplies no authority to change the verified vendor record. Gate resolves the vendor against master data, identifies the new

account as unverified and blocks payment to it. The change is routed to a specific verification obligation, such as a callback through a contact channel already present in the trusted vendor record. A phone number supplied in the suspicious email cannot satisfy it. Approval of the bank-detail update, when it comes, is not approval of a payment.

6.5 Behavior when dependencies fail

Governance that disappears during an outage is not governance. The plan specifies dependency-specific degraded behavior rather than a single rule, so operators know in advance what continues and what stops.

Table 5. Selected degraded-operation behaviors

If this is unavailable	The platform does this
Council	No new approvals or governance activations. Previously authorized work proceeds only if its obligations and validity windows still hold.
Warden	No new execution grants. Unconsumed grants follow the qualified validation and revocation rules.
Server or the mandatory journal	No new consequential dispatch that requires those records. In-flight uncertainty is preserved and recovered.
Registry	Verified cached artifacts may be used only within explicit freshness and revocation limits; otherwise the action is refused.
Sentinel	Hard authorization and budget controls keep working. Monitoring gaps are visible, and actions that require current monitoring stop.
A connector (timeout or crash)	The claim moves to an investigated state. No other worker blindly repeats the call.

7 Governing change itself

Most governance failures in automated systems are not dramatic. They are quiet changes: a tool scope widened for a pilot, an approval rule relaxed during a busy quarter, an environment switched from enforce to observe and never switched back. The platform treats every such change as a governed event in its own right, with the same separation of powers as an external action.

7.1 Propose, ratify, activate

A governance change moves through explicit states: candidate, tested, ratified, scheduled, activated and eventually superseded. The record links the old and new artifact digests, the proposer, the approver, the expected current activation and the reason. **Proposal, ratification and activation are three separately held powers.** A principal cannot ratify a change that policy identifies as its own, and an agent-derived identity cannot become the ratifier by delegating through another service.

Before a change is ratified, shadow evaluation replays a declared set of historical proposals under the candidate rules and reports which decisions would change. That impact report is valuable, but the plan is careful about its limits: it describes the evaluated set, not all future behavior. Adversarial and boundary tests are kept independently of recent production traffic so a quiet quarter does not become evidence that a weaker rule is safe.

7.2 Weakening a control is itself a governance change

Any change that widens a tool, alters an approver role, disables required evidence or moves an environment from enforce to observe must pass through the same lifecycle, optionally with a time-lock for review. Rollback is not an edit: it is a new activation of a known prior artifact. Emergency rollback can be pre-authorized for a specific previously approved digest within a bounded scope, but an agent cannot label an arbitrary new bundle a rollback.

7.3 Solving the bootstrap problem honestly

Council depends on the foundation, yet the foundation's governance transitions need Council's attestation. The plan breaks that cycle with an explicit, temporary bootstrap: before Council is qualified, Server accepts only a narrowly defined attestation from an owner-provisioned, non-agent authority key, bound to the intended transition and its expected prior state. Its scope, expiry and audit record are part of the installation, and a recorded trust transition retires it once Council is ready. It is a documented path, not an undocumented bypass, and no agent ever holds it.

A LIMIT STATED PLAINLY

One founder is not two independent reviewers

The platform will support approval quorums of several people. The founder's one-person organization cannot demonstrate that internally. Where a release or deployment requires genuinely independent human review, an external reviewer or the customer's own authority must participate, and until then the corresponding claim remains unqualified. A second model's review is recorded as additional analysis, never as a second accountable person.

8 Evidence and assurance

An organization that cannot reconstruct what its agents did cannot defend what they did. The assurance plane exists so that engineers investigating a failure and assessors reviewing a control can work from the same verifiable material, without trusting the interface that produced it.

8.1 Sentinel: useful before sophisticated

Sentinel's first release shows a task's sequence of proposals, decisions, approvals, claims, dispatches, receipts and unresolved outcomes, with the exact evidence and policy versions observed at each decision. Basic counters, including denial and escalation rates, missing receipts, budget saturation and connector drift, come before anomaly models. An alert on an impossible state transition is worth more on day one than a clever detector built on an unreliable event contract.

Sentinel can request a pre-authorized suspension through Warden. It can narrow capability; it cannot restore broader authority because an anomaly score falls. The measured property that matters is the time from an authenticated suspension request to the refusal of new affected grants. A dashboard that shows *suspended* while the action path still accepts work has failed its most important contract.

8.2 Assure: evidence that can be checked elsewhere

Assure produces a portable evidence package: the deployment boundary, component composition, time interval, ledger ranges, policy activations, approvals, exceptions, incidents, unresolved outcomes and declared omissions, each with a digest and a link to the authoritative record. A verifier checks the package without access to the running Console. A package with a missing receipt reports the gap; integrity verification never turns incomplete evidence into proof of completeness.

8.3 Console: one interface, no hidden privilege

Console gives approvers the exact proposed effect, target, request digest, evidence, obligations, expiry and relevant prior state, and it distinguishes a new approval from a stale request that no longer matches its target or policy. It separates proposed, authorized, dispatched, completed and unresolved rather than rendering every non-error response as success. Every consequential Console action calls a public governed API, so the interface never becomes a privileged side door.

8.4 Alignment with the frameworks your auditors use

Assure maps control objectives to evidence types and states the limits of each mapping. The mapping below is an evidence-planning aid, not a legal applicability assessment or a certification claim; regulatory applicability and interpretation remain the adopting organization's responsibility.

Table 6. Framework alignment: what the platform can evidence and what remains yours

Reference	Evidence the platform is designed to supply	What remains outside the claim
NIST AI RMF (Govern, Map, Measure, Manage)	Ownership, inventory, policy decisions, measurement, incidents and response	Organizational risk management and business judgment

Reference	Evidence the platform is designed to supply	What remains outside the claim
NIST SP 800-207 (zero trust)	Per-request authorization, verified identity, scoped access, explicit boundaries	Proof that every enterprise system and admin path follows zero trust
ISO/IEC 42001 (AI management system)	Controlled records, reviews, responsibilities and operational evidence	Certification against the whole management-system standard
OWASP guidance for LLM applications	Tests addressing injection-driven effects, excessive agency and disclosure paths	Elimination of all model, application and supply-chain risk
EU AI Act, Articles 9, 12, 13, 14	Versioned risk controls, record-keeping, system descriptions, approval and oversight records	Determination of applicability and role; the quality of human oversight

One sentence in the plan deserves repeating for every governance program: **consistent enforcement does not make an inappropriate policy appropriate.** The enterprise still decides its risk appetite, approver roles, data classifications, lawful uses and acceptance of residual risk. The platform enforces the chosen, implemented control and preserves evidence that it operated.

9 The development perspective

9.1 One hub, nine repositories

The platform is organized as one public architecture hub and nine independent component repositories, alongside the existing foundation repositories. The hub carries the shared explanation, roadmap, cross-component decision records, normative interface specifications, golden test vectors, integration designs and, when available, evidence for a tested composition. It is deliberately **not** a runtime component: a deployed Registry or Council accepts only artifacts that pass its own admission and authority checks, so a GitHub outage or a compromised documentation branch can never become permission to activate new runtime capability.

Each component repository carries its implementation, tests, migrations, diagnostics, a local development recipe and release evidence, and its README states its current capability tier and what it does not support. As of this writing the nine repositories contain dependency-free interface scaffolds and build guides with a first bounded work packet each, such as validating a candidate manifest without activating it (Registry) or deterministic, decision-only evaluation and replay (Gate). Compiling a scaffold is explicitly not implementation evidence.

9.2 Contracts before code

Nine repositories create coordination cost, and the most expensive failure would be components that each invent slightly different meanings for a grant, a principal chain or a request hash. The plan therefore resolves shared semantics in hub decision records before parallel implementation begins, in dependency order: principal context, then manifest and activation, then the action proposal and its canonical form, then decisions and provenance snapshots, approvals, and the claim-and-grant protocol. Breaking changes follow an **expand, migrate, remove** sequence. A platform release is a composition manifest that lists exactly which component versions were tested together; matching version numbers or a floating main branch are never treated as compatibility evidence.

9.3 Invariants as the acceptance language

The hub maintains a catalog of twenty-two invariants, each naming a claim, its trust assumptions, its owner, the tests that exercise it and the release evidence. They read like plain-language control statements, which makes them useful well beyond this project. Examples include: ordinary governed-write authority cannot activate a governance profile; no qualified target credential is readable from the agent environment; approval is invalid after relevant content, policy or target preconditions change; an unresolved effect is never blindly repeated; and a release advertises only the profiles and capabilities its evidence supports. The full catalog appears in Appendix A as a checklist any team can adapt.

9.4 The founder-led roadmap

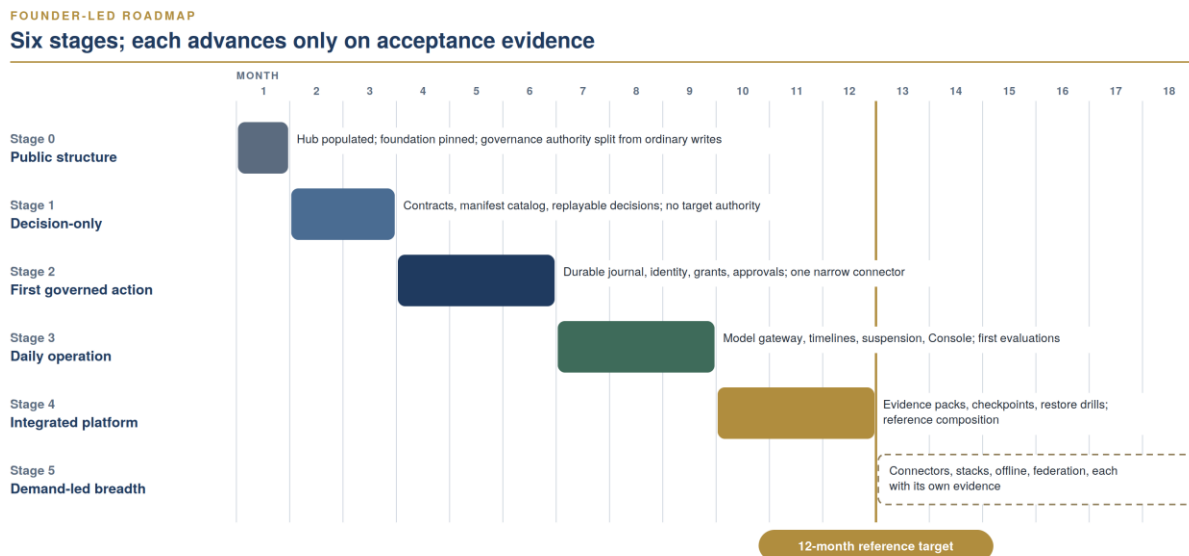


Figure 6. The founder-led roadmap. Month windows are capacity-based targets; the exit evidence for each stage is the commitment.

The roadmap advances in dependency order. Stage 0 establishes the public structure, pins and qualifies the foundation, and demonstrates refusal of an unauthorized governance

transition. Stage 1 delivers contracts and decision-only capability, where the platform can explain and replay a decision without claiming to govern real effects. Stage 2 produces the first complete governed action on one narrow connector, one identity system and one deployment profile, and it is where the most important proof point lands: the agent's environment contains no credential that can execute the governed action directly. Stages 3 and 4 add daily operational use, the human interface, evidence packs and recovery drills, culminating in a reproducible reference composition around month twelve. Stage 5 adds breadth only where demand, a test environment and a maintainer exist.

When capacity is constrained, scope is reduced in a fixed and published order: connector breadth, SDK breadth, interface polish, packaging variants and simultaneous adopter commitments first; advanced analytics, federation and multi-region automation next.

Request binding, mandatory evidence, credential isolation and required distinct authority are never removed to preserve a date.

9.5 What it costs to build in the open

Removing payroll from the critical path does not make the program free. The plan publishes an illustrative annual direct-cost envelope, staged so that nothing is spent before its gate is reached. It excludes founder compensation, which requires a separate runway calculation.

Table 7. Illustrative annual direct-cost envelope (planning example, not a quotation)

Direct project cost	Illustrative allowance
Coding-agent subscriptions and model or API use	\$48,000
Cloud, CI, storage and test environments	\$18,000
Independent security and identity review	\$35,000
Equipment, tools, legal, licensing and accounting	\$20,000
Demonstrations, workshops and travel	\$5,000
Contingency	\$24,000
Total illustrative envelope	\$150,000

The largest single line after model use is independent review. That is intentional: a lower-spend exploration phase can limit model usage and use disposable local targets, but high-consequence production claims wait for the review their risk class requires. Reduced cash narrows the supported scope; it does not turn assurance into a missing line item.

10 Building it with VCP

10.1 What VCP is

Vibe Code Pro (VCP) is Ioka's open-source experiment in a local coding agent. It combines repository-aware coding, model selection through OpenRouter, persistent governed memory derived from Munarium, visible delegated work and explicit cost accounting. Its engine is derived from a pinned copy of the open-source Codex engine, with selected behavioral adaptations from Gemini CLI and selected Munarium memory code, and its first planned release is a native Windows command-line application. Like the platform, VCP publishes its status honestly: its canonical host, durable commands and shared accounting have tested boundaries, while the installable application remains planned.

VCP matters to this paper for two reasons. It is the founder's intended primary work environment for building the platform, and its design already embodies the platform's principles at the scale of a single developer. One engine owns task state, authorization, the canonical store and cost accounting; every model request passes capability filtering and atomic cost admission; attributed inputs cannot grant authority; and a missing prerequisite is recorded as not run, never as passed.

10.2 How VCP will be used

BUILDING WITH VCP

Agents produce work and evidence; the founder holds acceptance and release

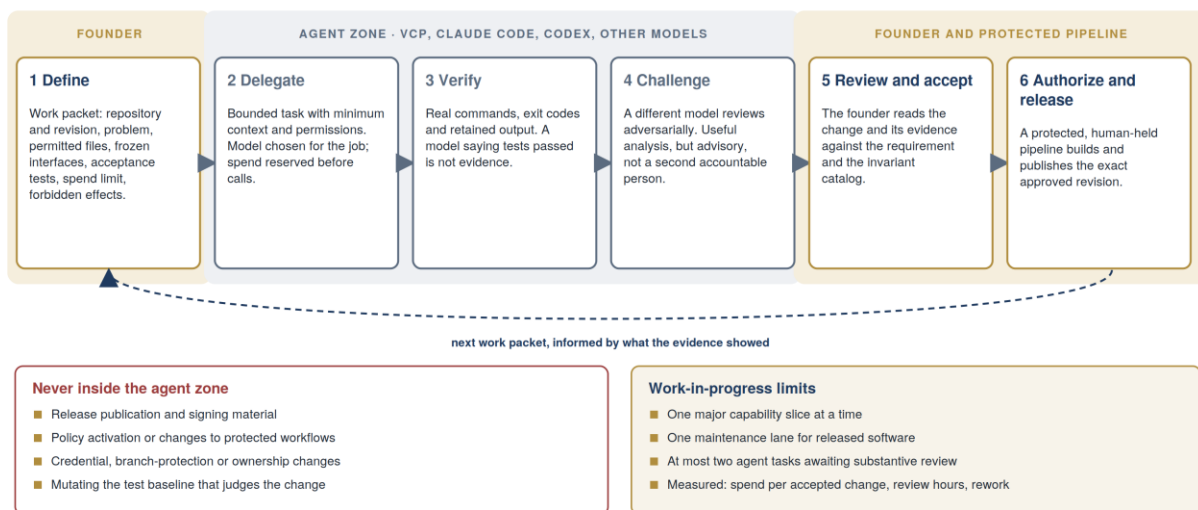


Figure 7. The development loop. Coding agents define nothing and release nothing; they deliver bounded changes with retained evidence for the founder to accept.

Work is organized into **work packets**. A packet names the repository and revision, the problem, an approved design or a bounded design question, the files an agent may touch,

the interfaces it must not change, the acceptance tests, and a time or model-spend limit. Security-sensitive packets also list forbidden effects: release publication, policy activation, credential changes, modifications to protected workflows or mutation of the reference test baseline.

Within a packet, VCP routes work to the model best suited to it, reserves spend before calls and keeps a durable record of what was attempted. Implementation and adversarial review may use different models, but both remain advisory. A model's statement that tests passed must point to real commands, exit status and retained output. The founder then reads the change and its evidence, and only a protected, human-held pipeline builds and publishes the exact approved revision.

VCP is the primary tool, not the only one. Claude Code, Codex and other agents remain available when they fit a task better, and work stays portable between tools through checked-in specifications, issues, reproducible scripts and evidence. VCP should reduce friction; it must not become a single dependency capable of stopping all platform development.

10.3 Why this method: the product and the process are the same idea

ONE PHILOSOPHY, TWO PLACES

The way the platform is built mirrors the way it governs



Figure 8. The development method mirrors the product. Each discipline VCP applies to coding agents has a direct counterpart in how Munarium governs enterprise agents.

Figure 8 is the strongest argument for building this way. The platform claims that agents can do substantial, valuable work without holding the authority to redefine their controls, approve their own effects or turn an incomplete test record into a release claim. The development process tests that claim every day. If bounded coding agents cannot build a

governance platform under those rules, the platform's premise is suspect; if they can, the project becomes its own first reference case, and Section 12's software case study describes exactly that pattern for other organizations.

10.4 Guardrails on the method itself

- **Work-in-progress limits.** The starting limit is one major capability slice, one maintenance lane for released software and no more than two bounded agent tasks awaiting substantive review, so agent throughput never creates a review backlog one person cannot resolve.
- **Decisions before parallel work.** Security and protocol questions that affect several repositories are settled in a hub decision record first, so agents cannot independently invent incompatible semantics.
- **Measured, not assumed, leverage.** The founder tracks model spend per accepted change, review hours, rework, escaped defects, security findings, restore success and elapsed time to useful capability. No blanket productivity multiplier is assumed.
- **No self-upgrading tools.** VCP's own changes are built and qualified separately from the platform releases they help produce. A running agent never upgrades its active enforcement code or silently changes the controls around its current task. A tool can help build its successor without ratifying it.
- **Accountable contributions.** Every AI-assisted contribution has a human submitter who signs off, discloses tools and borrowed material, and supplies executed test evidence.

11 Adoption: meeting enterprises where they are

11.1 Maturity is not a prerequisite

The platform is designed for organizations at every stage of agent adoption. A company does not need a mature agent platform to benefit from governed memory or an explicit model budget, and it should not wait for one. The control boundary grows with consequence.

ADOPTION

Start where your workload is; move the control boundary as consequence grows

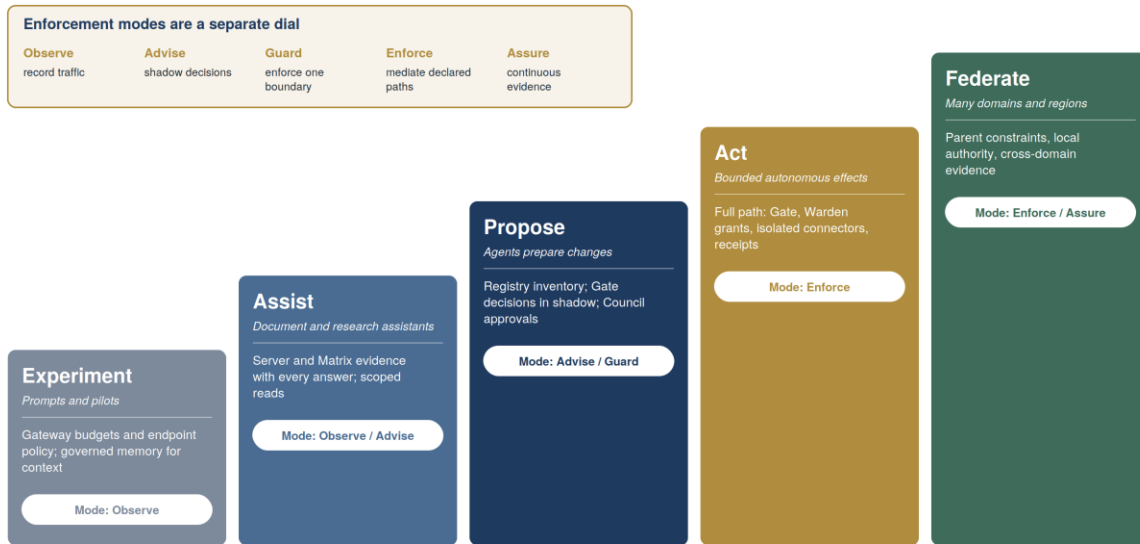


Figure 9. Adoption stages and enforcement modes. The entry point follows the workload and its consequence; enforcement mode is a separate dial recorded with every decision.

The five adoption stages are **experiment**, **assist**, **propose**, **act** and **federate**. The first two produce value without giving an agent any external action authority. At *propose*, the platform captures and evaluates requests while execution stays elsewhere. *Act* requires the complete authorized path for the chosen scope. *Federate* adds multiple authority and data domains, in which a local authority can add restrictions but never remove a mandatory parent prohibition.

Enforcement mode is a separate dimension: **observe** records traffic, **advise** computes shadow decisions, **guard** enforces a selected boundary, **enforce** mediates the declared consequential paths and **assure** adds continuing evidence and operational checks. Every record identifies the mode that actually applied. Observe and advise are valuable for building a baseline, but they do not make an unsafe path safe, and a dashboard toggle is not a control unless the capability is technically behind it.

11.2 Brownfield patterns

Few enterprises can replace their agent runtimes. The platform favors the smallest migration that removes a meaningful unmanaged capability, such as a single production deployment action, customer-message path or vendor-update operation, while other applications remain outside the pilot.

Table 8. Brownfield integration patterns

Pattern	How it is used	Boundary that must stay visible
Front-door mediation	Gate becomes the only approved path to a narrow API or tool.	Credentials and network routes outside that path must be removed or explicitly excluded.
Isolated adapter or sidecar	A legacy runtime's calls are converted into action proposals outside its process.	The agent must not be able to bypass or reconfigure the adapter.
Delegated enterprise workflow	The agent submits a request; an approved worker executes after Council records authority.	Workflow evidence and the exact execution request must remain bound.
Observation before migration	Metadata or traffic builds an inventory and a shadow-policy baseline.	Non-enforcing; must never be described as governed execution.

11.3 Many stacks, selective qualification

The architecture targets Microsoft-centered, AWS-centered, Google-centered, open or on-premises, and mixed legacy estates using the same authority contracts. The plan is equally clear that these are target profiles, not support claims. The first-year baseline qualifies a reproducible local reference stack and one sponsor- or adopter-backed enterprise profile; other stacks remain documented recipes or experimental adapters until someone can reproduce their qualification, maintain them and explain their failure modes. Integration priorities follow the same logic: identity, secrets, a durable record, the Action API and one application client come first; CI/CD, one approval integration, model routes and evidence export follow; ERP, CRM, document platforms and federation are adopter-driven.

12 Three forward-looking case studies

READ THIS FIRST

These cases are synthetic

The organizations below are fictional composites, and the timelines, baselines and targets are illustrative. They show how the platform is designed to be adopted once the relevant roadmap stages deliver their evidence. They are not customer claims, measured results or commitments of support. Each case begins as a design-partner evaluation on disposable or sandbox targets, which is how the platform plan itself sequences adoption.

THREE SYNTHETIC CASE STUDIES

Different industries, the same pattern: start narrow, prove the boundary, then widen

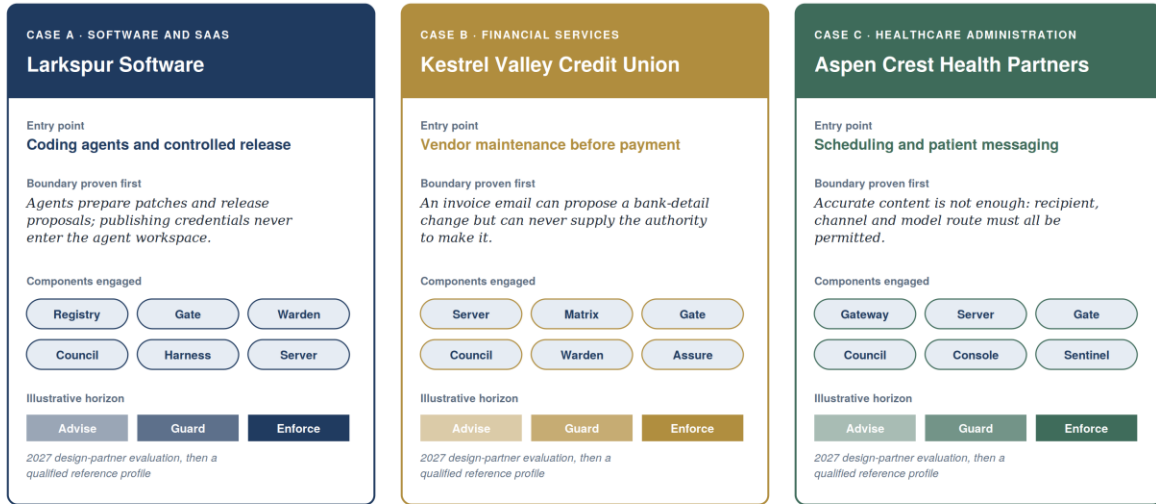


Figure 10. Three synthetic case studies. Each starts in a narrow workflow, proves one consequential boundary and widens only on evidence.

Case A. Larkspur Software: coding agents and a controlled release path

Profile	
Industry	B2B software-as-a-service; about 350 engineers across twelve product teams
Starting point	Coding agents already open pull requests in most repositories; CI tokens are broad and long-lived
Entry workflow	Agent-prepared release proposals for internal packages, published to a staging registry
Consequence	Package publication and deployment treated as C3; draft pull requests as C1
Horizon	Design-partner evaluation from roadmap Stage 1, enforcement after Stage 2, evidence packs after Stage 4

The situation

Larkspur adopted coding agents quickly and happily. Agents fix flaky tests, update dependencies and draft release notes, and developer satisfaction is high. A near-miss changed the conversation: during a routine dependency update, an agent's CI context carried a token with rights to publish packages to the company's registry. Nothing bad happened, but the security team could not say with confidence which agents could publish, which tokens they could see, or how a reviewer would know that the artifact released was the artifact approved. Larkspur's SOC 2 auditor had begun asking the same questions about change management.

How the platform engages

Larkspur starts where the platform itself starts, and where the plan names its preferred first reference case. In the first phase it registers its agent definitions and a single narrow tool manifest, *publish an approved artifact to staging*, in **Registry**, and runs **Gate** in advise mode against release proposals generated by its CI. Each proposal names an exact source revision, build-artifact digest, test evidence, target environment and change window. For a quarter, Gate simply explains what it would have decided. The shadow record surfaces two uncomfortable findings: several proposals referenced mutable download locations instead of digests, and some were approved by the same service identity that generated them.

Once the first complete governed action is qualified, Larkspur moves that one path to enforce. Publishing credentials leave every agent workspace and CI job that runs untrusted code. **Warden** brokers a single-use grant to an isolated publishing connector only after **Council** records approval from a distinct human release manager, bound to the artifact digest. Release freezes become a governed input rather than a sentence in a prompt. **Harness** gives developers typed outcomes, so an agent sees *approval required* or *unresolved* instead of a generic error it might retry.

What Larkspur measures

Table 9. Case A illustrative baselines and twelve-month targets

Measure	Assumed baseline	Illustrative target
Agent-reachable publish or deploy credentials in the qualified boundary	Several broad CI tokens	None
Release proposals bound to an exact artifact digest	Inconsistent	All proposals on the governed path
Required bypass tests (substituted artifact, changed target, stale approval, self-ratification, direct credential)	Not tested	All refused and recorded
Engineer time to assemble a release proposal	About half a day	Under an hour, agent-prepared
Time to produce change-management evidence for an audit sample	Two to three days	Same day, from an evidence pack

The boundary kept honest

Larkspur's evidence pack states what the pilot does not cover: developer laptops, other CI systems and production deployment, which remains a separate capability awaiting independent identity review. That candor is what makes the pack useful to an auditor.

Case B. Kestrel Valley Credit Union: vendor maintenance before payment

Profile	
Industry	Regional credit union; accounts-payable team processing several thousand vendor invoices a month
Starting point	An AI assistant already summarizes invoices; business email compromise attempts are a known threat
Entry workflow	Vendor-master bank-detail changes proposed from invoice evidence
Consequence	Vendor bank-detail changes and payment release are separate C3 capabilities
Horizon	Governed memory immediately; shadow decisions from Stage 1; one enforced update path after Stage 2

The situation

Kestrel Valley's finance leaders want AI help in accounts payable, where invoice volume grows faster than headcount. Their fraud team has one firm condition: no automation may ever make it easier for a spoofed email to redirect a payment. Examiners want the controls evidenced.

How the platform engages

Invoices and vendor correspondence become governed claims in **Munarium Server**, each tagged with its source and trust scope, so an email is recorded as external evidence rather than as a fact about the vendor. **Matrix** provides reviewed, read-only queries against the vendor master, so the assistant can compare an invoice's banking details with the verified record without any write path.

Next, the agent begins proposing vendor-master changes, and **Gate** evaluates them in advise mode across ninety days of historical change requests. The shadow report shows exactly which changes would have been classified C3 because the destination account was unverified. When the governed path is qualified, the credit union moves the single vendor-update operation behind Gate. The decision attaches a verification obligation: a callback through a contact channel already present in the trusted vendor record. A number in the suspicious email cannot satisfy it. **Council** then approves the exact update, and **Warden** brokers only the connector capability for that change. Payment release remains a separate capability that is simply not enabled in the pilot, and approval of a bank-detail change is never treated as approval of a payment.

For examiners, **Assure** produces evidence packs that show each proposed change, the verification recorded, the approver, the target record version and any unresolved outcomes, with declared omissions. It evidences the process, not compliance.

What Kestrel Valley measures

Table 10. Case B illustrative baselines and twelve-month targets

Measure	Assumed baseline	Illustrative target
Bank-detail changes with a recorded verification through a trusted channel	Manual and inconsistently documented	All changes on the governed path
Unverified destinations reaching an approvable state	Unknown	None; blocked with a typed reason
Verification minutes per legitimate change (burden on AP staff)	Not measured	Measured and reported alongside fraud controls
Time to assemble examiner evidence for a sample of changes	About a week	One to two days
Agent access to ERP or payment credentials	Under discussion	None

The boundary kept honest

A faster workflow is accepted only if the speed does not come from removing the verification step that gives the change its authority. The platform will not detect every fraudulent instruction; it ensures that untrusted content cannot supply the authority to act on one.

Case C. Aspen Crest Health Partners: scheduling and patient messaging

Profile	
Industry	Regional healthcare network; fourteen outpatient clinics and a shared contact center
Starting point	Staff use a document assistant for policies; rescheduling calls overwhelm the contact center
Entry workflow	Agent-proposed administrative schedule changes and patient messages, using synthetic records first
Consequence	Patient messages C2 with recipient binding; restricted exports C3 and disabled in the pilot
Horizon	Gateway data controls from Stage 3; enforced messaging path after qualification; production only with the network's own review

The situation

Aspen Crest wants an agent to help schedulers by proposing appointment changes and drafting the messages that go with them. Its privacy office has two concerns. First, an accurate message sent to the wrong person, or through an unapproved channel, is a disclosure. Second, prompts containing patient information must never reach a model endpoint the network has not approved.

How the platform engages

Gateway comes first. Every model call is routed through it, with an endpoint allowlist tied to data classification: prompts that carry patient information may go only to approved endpoints, such as a local model or a hosted service covered by the network's agreements. Budgets are reserved per task, so a burst of rescheduling does not become a surprise invoice. A local retrieval index does not make a later hosted-model request local, and the reference documentation states exactly which data leaves the deployment.

The messaging path runs through **Gate** with a narrow connector. Each proposed message is checked against the approved patient context, recipient, channel, content classification and any required consent or authorization record that the network defines. A recipient changed after approval invalidates the approval. Schedulers review proposals in **Console**, which shows the exact message, recipient and evidence side by side. **Sentinel** tracks denial rates and cross-patient access attempts and can request suspension of the messaging capability if patterns change. The pilot uses synthetic records and test recipients until the network's own privacy and security review approves a production boundary.

What Aspen Crest measures

Table 11. Case C illustrative baselines and pilot targets

Measure	Assumed baseline	Illustrative target
Prompts with patient information reaching an unapproved model endpoint	Not controllable	None; refused at Gateway
Cross-patient access or changed-recipient attempts	Not observable	All refused and recorded
Scheduler time per rescheduling request	Several minutes	Reduced; reported with approval rates
Measured time from suspension request to refusal of new messages	Not applicable	Published for the qualified profile

The boundary kept honest

The platform does not determine clinical appropriateness or legal permission. Aspen Crest defines those rules; the platform enforces the rules it is given and preserves evidence that it did.

What the three cases have in common

Each organization started with a workflow it already cared about rather than a platform rollout. Each used observe or advise mode to learn where its real exposure was before enforcing anything. Each proved one consequential boundary, most importantly that the agent environment held no credential capable of the governed effect, before widening scope. And each accepted an evidence pack that states its own omissions, because that is what makes the evidence credible to someone outside the project.

13 Getting a jump on applied AI governance

Most of the value in this architecture does not depend on waiting for the platform. The practices below can be adopted now with the tools an organization already has, and each one prepares the ground for a component of the platform if and when the organization chooses to deploy it.

Table 12. Twelve actions to take now

Action	Why it matters	Prepares for
Inventory every agent, tool and credential, each with a named owner	You cannot govern what you cannot list, and discovered capability is not approved capability.	Registry
Assign each tool a consequence class from C0 to C4	Controls should scale with what an action can do, not how it is described.	Gate policy

Action	Why it matters	Prepares for
Remove ambient target credentials from agent runtimes	The single largest risk reduction available for agentic systems.	Warden
Separate propose, approve and activate for prompts, policies and tool scopes	Prevents quiet widening of authority and self-approval.	Council
Bind approvals to the exact request and give them an expiry	A changed recipient, amount or artifact should require a new approval.	Council, Gate
Treat unknown outcomes as unresolved and forbid blind retries	Prevents duplicate payments, tickets and deployments.	Gate, Harness
Route every model call through one gateway with budgets and endpoint rules	Controls cost and data disclosure in one place.	Gateway
Keep an append-only record of claims, decisions and evidence	Makes answers and actions reconstructable after the fact.	Server (released)
Put structured-data reads behind reviewed, read-only queries	Stops agents from composing arbitrary queries against sensitive data.	Matrix (released)
Run one high-value workflow in observe or advise mode	Reveals real exposure before any enforcement friction.	Adoption path
Write your invariants as tests and fail releases that lack evidence	Turns governance statements into checks that run.	Invariant catalog
Map evidence to NIST AI RMF, ISO/IEC 42001 and applicable EU AI Act duties	Reuses one evidence stream for several audiences.	Assure

13.1 A ninety-day starter plan

Table 13. A pragmatic first ninety days

Window	Focus	Outcome
Days 1 to 30	Discover. Inventory agents, tools, credentials and owners; classify consequence; identify the one workflow where an agent's mistake would hurt most.	An owned inventory and a shortlist of C2 and C3 capabilities
Days 31 to 60	Design the boundary. For the chosen workflow, write the narrow tool manifest, the approval rule and the evidence required; remove the agent's direct credential.	A documented control boundary and a credential removed
Days 61 to 90	Prove it in shadow. Record proposals and decisions without enforcement; run the bypass tests; publish an internal evidence summary that lists what is not covered.	A baseline, a set of refusals and an honest gap list

14 How to engage with the open-source program

The platform is built by one accountable founder, and the project is explicit that outside participation strengthens it without being counted as delivery capacity. That makes the terms of engagement unusually clear.

Table 14. Ways to participate

Role	What you provide	What you get
Self-service evaluator	Time to read the design, run releases as they appear and file reproducible issues	Early insight into the architecture and a voice in contract decisions
Design partner (at most two active at once)	An owner, a test environment, a narrow workflow, a review cadence and permission to share sanitized lessons	A workflow shaped around your boundary and direct design sessions
Sponsor	Funding for a named milestone, public adapter, test environment or independent review	Faster delivery of work you need; the resulting Ioka-owned code stays public
Contributor	Signed-off pull requests with tests and disclosure of any AI assistance or borrowed material	Recognition and influence over conformance fixtures, adapters and documentation
Independent reviewer	A scoped security, identity or assurance review of a stated revision	Named, recorded findings that advance a release label

Ioka LLC also offers bounded architecture reviews, implementation assistance, training and sponsored development consistent with actual capacity. Customers pay for expertise, delivery commitments or assurance services, never for permission to inspect or use essential controls. The first commercial constraint is capacity, not a paywall. Enquiries go to info@ioka.io, and the architecture hub is at github.com/iokaio/munarium-platform.

15 Conclusion

The Munarium Governance Platform is an attempt to answer a question every organization deploying agents will face: how do you let AI systems do consequential work without letting them hold the keys, approve themselves or rewrite the rules? The answer offered here is architectural rather than aspirational. Separate the four powers. Put controls on the paths where effects actually happen. Bind approvals to exact requests. Record every decision so someone outside the system can check it. Say plainly what is not covered.

The objective is not to show that one person can generate the largest possible codebase. It is to show that a single accountable founder, working with capable coding agents under the same discipline the product enforces, can progressively deliver a platform whose claims

remain inspectable as its capability grows. All nine components begin in the open. Useful releases arrive before the whole vision is finished. Broader enterprise support follows tested demand and available capacity.

For readers who want a head start, the most valuable step does not require any software at all: find the agent in your organization that holds a credential it should not, and move that credential somewhere the agent can only ask for it.

ACKNOWLEDGMENT

The founder gratefully acknowledges Jamey Kistner of OSINTelligence LLC for an exchange that sharpened the platform's treatment of the boundary between an agent's governed writes and the authority that governs them. Kistner's *The Sovereign Stack: Architecture, Discipline, and Evidence from One Desk* distinguishes real-time enforcement from verification outside a self-improving system's control. The acknowledgment does not imply endorsement, and Munarium Sentinel is a software assurance component, not a claim of equivalence to the hardware-isolated External Sentinel that work describes.

Appendix A. The invariant catalog as a checklist

The platform's twenty-two invariants are written as plain control statements, which makes them a practical checklist for any agentic system. In the platform, each will gain a test location, evidence artifact and last verified composition; today every evidence field is blank, meaning unverified.

Table 15. Invariant catalog

ID	Required property	Primary owner
INV-01	Ordinary governed-write authority cannot activate a governance profile.	Server, Council
INV-02	A discovered or agent-proposed tool remains inert until authorized activation.	Registry
INV-03	Every accepted action has a verified tenant and principal context.	Warden, Gate, Server
INV-04	Unknown or incompatible manifests fail closed.	Registry, Gate
INV-05	Canonical requests produce the same digest across supported clients.	Hub contracts, Harness
INV-06	Missing mandatory lineage does not become trusted authority.	Server, Gate
INV-07	No qualified target credential is readable from the agent environment.	Warden
INV-08	No grant is issued without a durable, matching execution claim.	Gate, Warden
INV-09	Approval is invalid after relevant content, policy validity or target preconditions change.	Council, Gate
INV-10	A grant cannot be concurrently consumed for multiple dispatches.	Warden, Gate
INV-11	An unresolved effect is never blindly repeated.	Gate, Harness, Console
INV-12	Revoked authority stops new affected work within the qualified bound.	Warden, Sentinel
INV-13	A delegated principal cannot widen its originating scope or become its own ratifier.	Warden, Council
INV-14	Required-recording failure prevents new consequential dispatch.	Server, Gate, Gateway
INV-15	Task and child-task reservations cannot exceed the enforced shared budget.	Gateway
INV-16	Console cannot perform a privileged operation unavailable through governed APIs.	Console, Council, Warden
INV-17	Missing evidence or telemetry is reported as a gap, not a successful interval.	Sentinel, Assure
INV-18	Evidence-package verification detects changed or missing required artifacts.	Assure, Server

ID	Required property	Primary owner
INV-19	Restore cannot silently reactivate a consumed grant or erase an unresolved claim.	Gate, Warden, Server
INV-20	A local overlay cannot waive a mandatory parent prohibition.	Council, Registry, Gate
INV-21	Untrusted pull-request code cannot acquire release secrets or replace active approval controls.	Hub and repository CI
INV-22	A release advertises only the profiles and capabilities supported by its evidence.	All maintainers

Appendix B. Working glossary

Term	Meaning in this paper
Action proposal	A typed request for a specified effect, submitted without target execution authority.
Activation	The distinct power to make a new policy, manifest or profile effective.
Composition	The exact set of component and contract versions tested together for a defined deployment profile.
Consequence class	The Co to C4 classification derived from the approved manifest and verified context.
Durable claim	Persistent execution state recorded before any grant or dispatch.
Evidence package	A verifiable collection of linked records, artifacts, boundaries and declared gaps.
Execution grant	Request-bound, time-limited authority consumed once through the platform's execution protocol.
Governed write	A proposed memory claim or correction evaluated under the active governance rules.
Ratification	Approval of a governance change by an authority distinct from its proposer.
Unmanaged path	A route to data or an effect outside the qualified mediation boundary.
Unresolved outcome	An effect whose result is unknown and must be investigated rather than repeated.
Work packet	A bounded unit of agent-assisted development with permitted scope, tests, limits and forbidden effects.

Appendix C. Sources

1. Ioka LLC. *Munarium Governance Platform: A Founder-Led Open-Source Architecture and Delivery Plan*, revision 4, 28 September 2026. Published in the architecture hub, github.com/iokaio/munarium-platform.

2. Ioka LLC. Munarium Governance Platform architecture hub: overview, component catalog, invariant catalog, release labels, roadmap, governance model, build plan, system boundaries, contract backlog and foundation requirements. github.com/iokaio/munarium-platform, accessed 29 September 2026.
3. The Munarium founder. *The Governed Must Not Govern the Governor*. Ioka LLC position paper, September 2026.
4. Ioka LLC. *VCP: Vibe Code Pro*, project overview and delivery contract. github.com/iokaio/vcp, accessed 29 September 2026.
5. Ioka LLC. Munarium Server and Munarium Matrix. github.com/iokaio/munarium and github.com/iokaio/munarium-matrix.
6. Kistner, J. *The Sovereign Stack: Architecture, Discipline, and Evidence from One Desk*. OSINTelligence LLC, 2026. Concept DOI 10.5281/zenodo.22316158.
7. National Institute of Standards and Technology. *AI Risk Management Framework; SP 800-207: Zero Trust Architecture*.
8. International Organization for Standardization. *ISO/IEC 42001:2023, Artificial intelligence management systems*.
9. European Union. *Regulation (EU) 2024/1689* (the AI Act).
10. OWASP GenAI Security Project. *Risks and guidance for LLM applications*.
11. IETF. *RFC 8693: OAuth 2.0 Token Exchange; RFC 9449: OAuth 2.0 Demonstrating Proof of Possession*.
12. Model Context Protocol. *Authorization* specification.